

How machines learn to see

One day, one story

If you can multiply small numbers, you can follow every step today. New words are only used AFTER we earn them.

- **Part 1 — The trick.** How a pile of numbers becomes the word “cat”. We compute it by hand.
- **Part 2 — The learning.** Nobody programs it. It practices, like the hot-and-cold game.
- **Part 3 — The shortcut.** Borrow a model someone already trained on a million photos.
- **Part 4 — The superpowers.** Find objects, cut them out, talk to images, create images.
- **Part 5 — The cheat sheets.** Every term and every knob, on slides you keep.

Five words we will use all day

Only these five up front. Every other term gets introduced when you have already understood the idea behind it.

WORD	WHAT IT MEANS, IN PLAIN WORDS
MODEL	a machine made of numbers: input goes in, a guess comes out
DATASET	the pile of examples we teach with — photos plus their correct answers
LABEL	the correct answer attached to one example: “this photo is a cat”
TRAINING	showing the model examples, over and over, until its guesses get good
ACCURACY	out of 100 tries, how many guesses were right

A computer has never
seen a cat.

A photo is just a grid of numbers

- Zoom far enough into any photo and it falls apart into little squares.
- Each square holds **one number**: 0 means black, 255 means white, in-between means gray.
- A color photo is **three grids stacked** — one for red, one for green, one for blue. That is ALL the computer gets.

THE PROPER NAME

PIXEL — one little square of the photo

TENSOR — a grid of numbers (possibly several stacked) — nothing scarier than that



Chelsea — a real photo, with a tiny square marked on her eye.
Below: the REAL numbers inside that square.

67	59	53	51	48	38	21
64	68	61	58	49	28	16
73	70	70	63	46	20	12
86	91	84	67	48	25	17
95	91	89	65	53	50	42

The trick, by hand — it is just times tables

The image has a bright stripe (2s) next to a dark area (0s). Where they meet is an edge. Can math FIND that edge?

1 · A TINY IMAGE

2	2	0	0	0
2	2	0	0	0
2	2	0	0	0
2	2	0	0	0
2	2	0	0	0

Bright stripe of 2s, dark area of 0s. The bold box is where we lay the stencil first.

2 · A 3×3 STENCIL

1	0	-1
1	0	-1
1	0	-1

Nine chosen numbers. Plus on the left, minus on the right.

3 · MULTIPLY MATCHING CELLS, ADD ALL NINE

$$\begin{array}{rcl} 2 \times 1 + 2 \times 0 + 0 \times (-1) & = & 2 \\ 2 \times 1 + 2 \times 0 + 0 \times (-1) & = & 2 \\ 2 \times 1 + 2 \times 0 + 0 \times (-1) & = & 2 \end{array}$$

$$\text{total} = 6 \rightarrow \text{“my pattern is HERE”}$$

Now slide the stencil onto the flat dark area: every cell is 0, every product is 0, total = 0 — “nothing here.” The stencil only speaks on its own pattern.

THE PROPER NAME

CONVOLUTION — this multiply-and-add operation — the heart of all computer vision

FILTER (or KERNEL) — the little stencil of numbers

Slide it everywhere — the photo answers back

- Slide the SAME stencil across the whole photo — thousands of little multiply-and-adds.
- Write the answer at every stop. You get a new image that **glows where the pattern lives**.
- Look: the whiskers and eye outlines light up — they are vertical-ish edges.
- This is real output — this exact stencil ran on this exact photo.

THE PROPER NAME

FEATURE MAP — the glow-map an image produces for one filter: bright = pattern found here



The photo, in grays



Our vertical-edge stencil: whiskers glow



Same stencil rotated 90°: horizontal edges glow

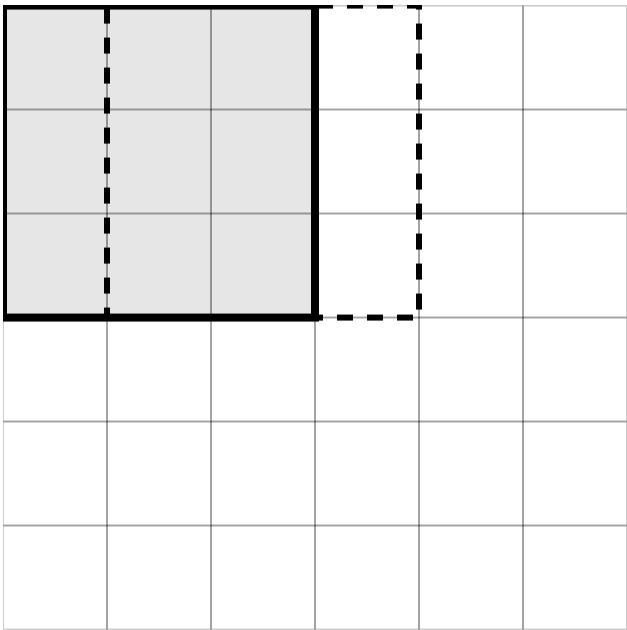
TURN THE KNOB

Different numbers in the stencil → a different pattern found. Rotate it → horizontal edges. The numbers ARE the pattern.

Two sliding rules: stride and padding

Both are just rules about HOW the stencil walks across the photo.

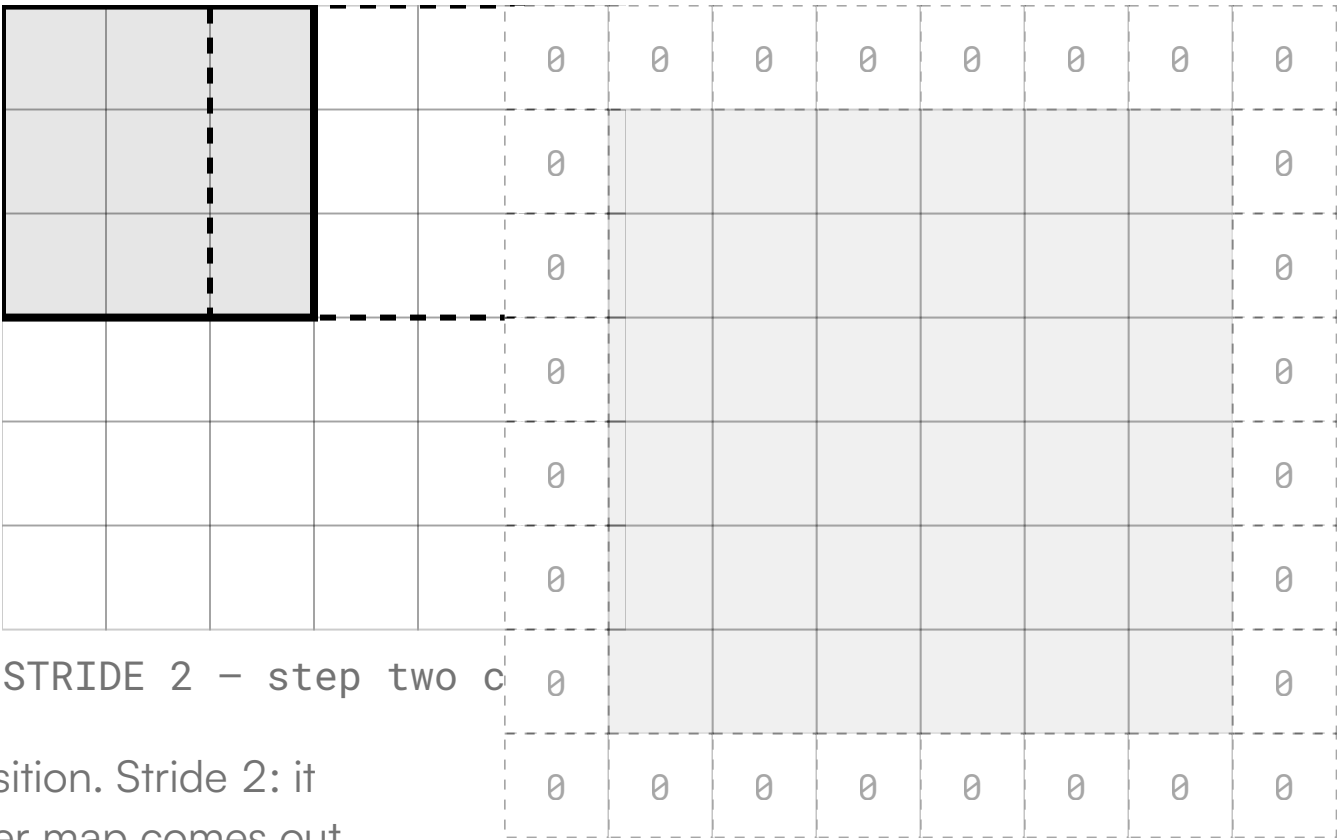
STRIDE – THE STEP SIZE



STRIDE 1 – step one cell

Stride 1: the stencil visits every position. Stride 2: it skips every other one — the answer map comes out half the size.

PADDING – ZEROS GLUED AROUND THE BORDER



STRIDE 2 – step two cells

Without padding the border pixels never sit under the stencil’s center, and the map shrinks each layer. The dashed ring of invisible zeros fixes both.

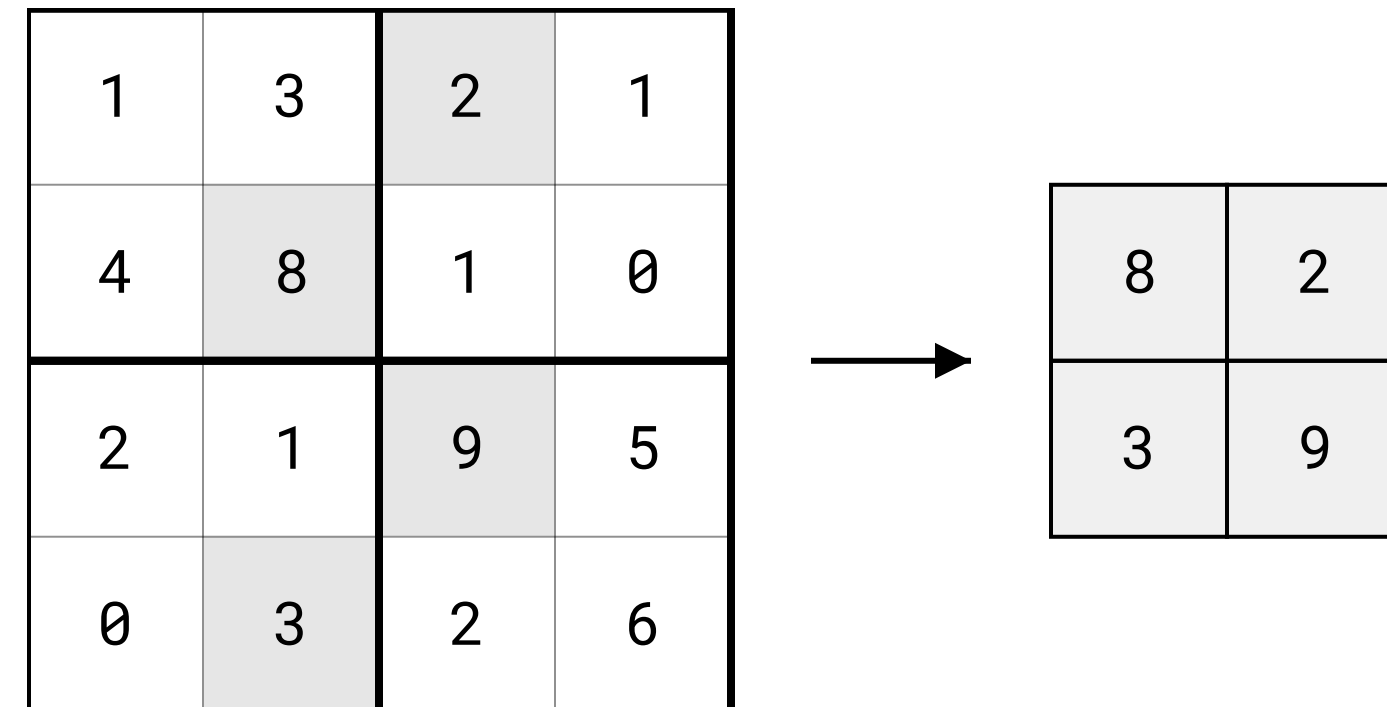
TURN THE KNOB

- stride 1** → **2** → output map half the size; cheaper, but a blurrier view
- no padding** → a 3×3 stencil shrinks a 7×7 photo to 5×5 every layer
- padding 1** → the map stays the same size as the photo
- kernel 3×3** → **7×7** → sees bigger patterns at once, but 27 → 147 numbers to learn per filter — slower, needs more data

Pooling — shrink the map, forgive small shifts

- Look at each little 2×2 window and **keep only the biggest number**: “was the pattern here at all?”
- The map shrinks to half size → everything after it is $4 \times$ cheaper.
- If the pattern moves by one pixel, the biggest number in the window usually does not change → **small shifts stop mattering**.
- Cousin: **average pooling** takes the mean instead. At the very end, averaging each whole map down to ONE number gives a short summary list of the photo.

MAX POOLING, 2×2 — KEEP EACH QUADRANT'S WINNER



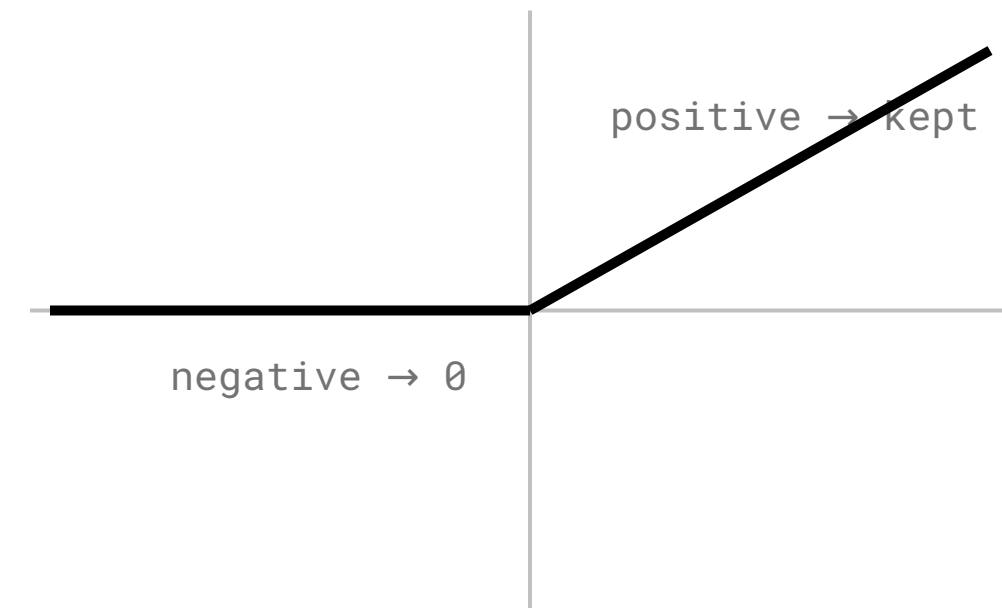
THE PROPER NAME

MAX / AVERAGE POOLING — keep the max (or the mean) of each small window

GLOBAL AVERAGE POOLING — average each whole map to one number — the final summary step

ReLU — the “keep the good news” switch

- After each convolution, walk over the map and **replace every negative number with 0**. Positives pass through untouched. That is the entire rule.
- Why bother? Stacking plain multiply-and-add layers is like stacking rulers — **a stack of rulers is still a ruler**. One bend between layers lets each layer build something NEW on top of the last.
- Without it, a 50-layer network computes nothing more than a 1-layer one.



THE PROPER NAME

ACTIVATION FUNCTION — the bend between layers; ReLU is the standard one

RELU — $\max(0, x)$ — negatives become 0, positives stay

SIGMOID / TANH — older bends that squash numbers into 0—1 or -1—1 — you will see the names

Stack it all — the complete seeing machine

Filters find edges. The next layer looks at the GLOW-MAPS and finds combinations: textures, then parts, then objects.



THE PROPER NAME

CONVOLUTIONAL NEURAL NETWORK — CNN — the whole stack of filter layers

EMBEDDING — the short list of numbers that summarizes the photo

IMAGE CLASSIFICATION — one label for the whole photo — what this machine does

AND THE LADDER IT CLIMBS

SOFTMAX — turns final scores into percentages that add to 100

EDGES → TEXTURES → PARTS → OBJECTS — what layers 1, 2, 3... learn to find, in that order

The whole “eye” is a few honest lines

Every line below is something you already understand. Read the comments, not the syntax.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1), # 32 stencils, 3x3, on R,G,B
    nn.ReLU(),                               # keep the good news
    nn.MaxPool2d(2),                          # shrink, forgive shifts
    nn.Conv2d(32, 64, kernel_size=3, padding=1), # 64 deeper stencils
    nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),                  # global average -> summary
    nn.Flatten(),
    nn.Linear(64, 10),                        # 10 class scores
)
```

A REAL, COMPLETE IMAGE CLASSIFIER - THIS SHAPE SCORES ~50% ON A 10-CLASS PHOTO TASK (CHANCE = 10%)

TURN THE KNOB

out_channels 32 → 64 → more patterns spotted per layer; slower, more to learn

kernel_size 3 → 7 → each stencil sees a bigger patch; weights jump 27 → 147 per filter

delete the ReLU lines → the stack collapses into one big ruler — accuracy tanks

add more conv blocks → deeper ladder: textures → parts → objects; needs more data and time

Close the laptop lids — quick check

Answers out loud, no grades. If the room can answer these, we have earned the next part.

A friend says “the computer looks at the photo.” What does it **ACTUALLY** get?

A grid of numbers, 0—255 — three grids for a color photo. Nothing else.

What does a filter do, in one sentence?

It slides over the photo, multiplies and adds, and writes a big number wherever its pattern appears.

Why does a CNN need ReLU between layers?

Without the bend, the whole stack collapses into one layer — stacked rulers are still a ruler.

What does pooling buy us?

Smaller maps (cheaper) and forgiveness for small shifts of the pattern.

MYTH VS REALITY

Myth: AI sees the way we do — it just “recognizes” the cat.

Reality: It has no eyes and no idea what a cat is. It does millions of multiply-and-adds on a number grid, and the numbers it multiplies by were learned, not programmed.

Nobody programs the filters. The network finds them.

On slide 6, WE chose the nine stencil numbers. In a real network, all the stencils — millions of numbers — start as random noise. Then they learn.

Learning is the hot-and-cold game

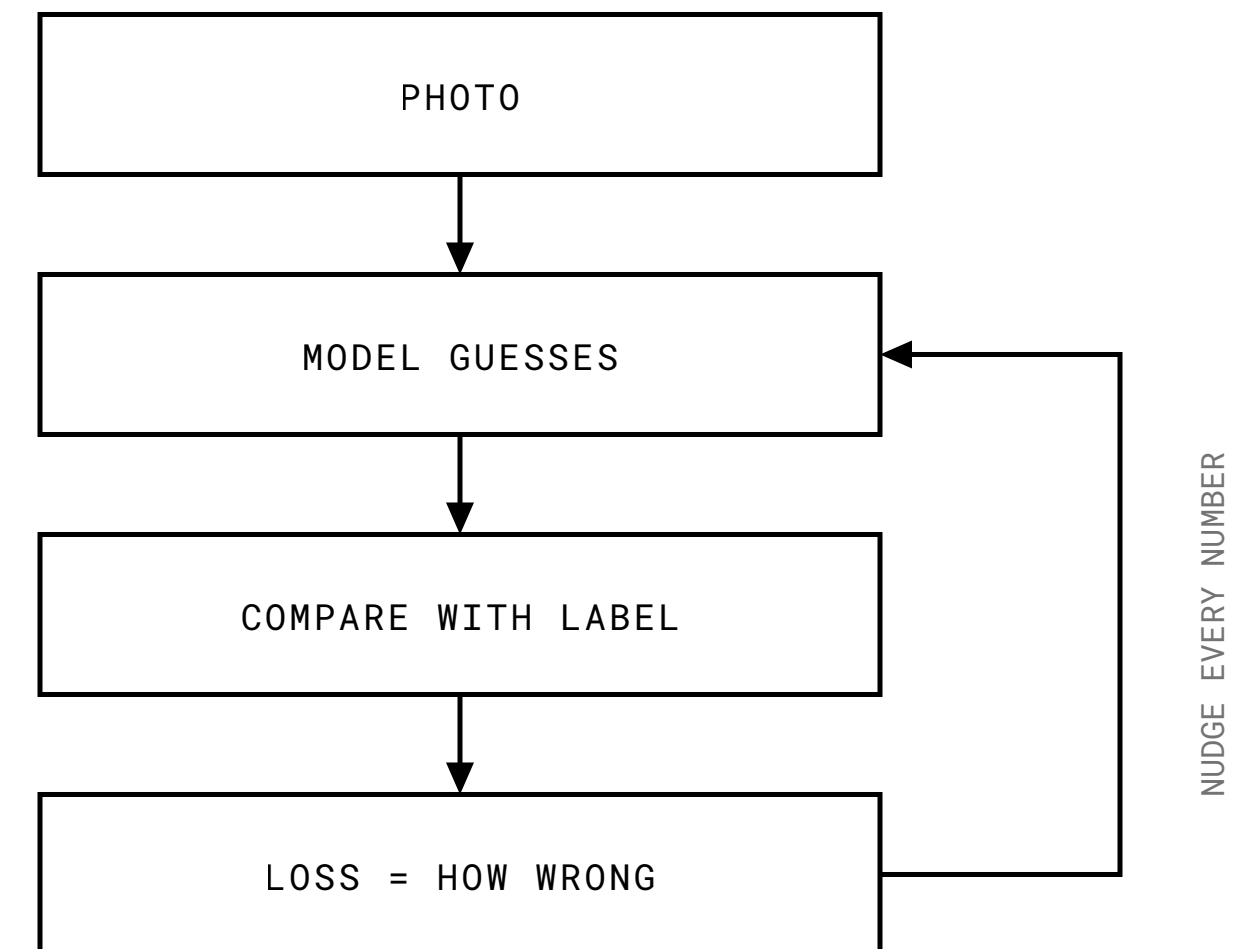
- Show the network a photo. It guesses: “70% dog, 20% cat...” — badly, because its stencils are random.
- We know the label is “cat”, so we hand it one number — **how wrong it was**. Big number = ice cold.
- A recipe then tells **every stencil number** which direction to nudge to be slightly less wrong.
- One photo, one tiny nudge. A million photos later, the random stencils have become the edge detectors you saw earlier. Nobody put them there.

THE PROPER NAME

LOSS — the wrongness score — that is the entire meaning of the word

BACKPROPAGATION — the recipe that works out each number’s nudge direction

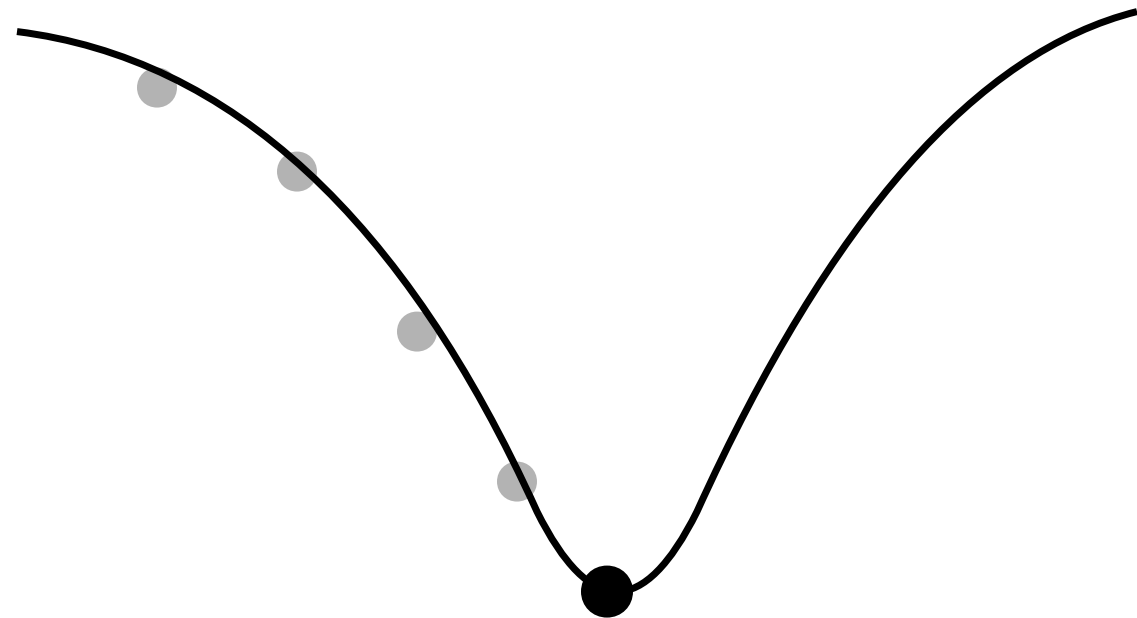
EPOCH — one full pass through the whole dataset



Wrongness is a landscape. Training rolls downhill.

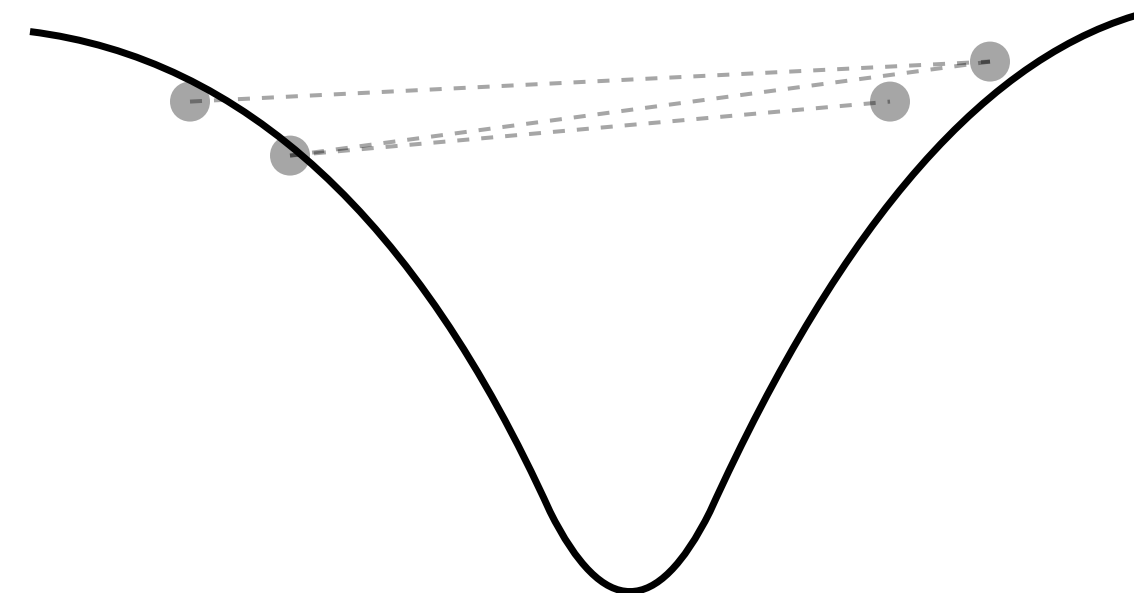
Imagine every possible setting of the stencil numbers as a point in a hilly landscape — height = loss.
Training = rolling the ball to a low spot.

GOOD STEP SIZE



downhill, step by step

STEP SIZE 10× TOO BIG



steps too big – bouncing across the valley

THE PROPER NAME

GRADIENT DESCENT — the rolling-downhill procedure

LEARNING RATE — the step size — the single most important knob in training

TURN THE KNOB

learning rate too small → the ball crawls; training takes forever

learning rate 10× too big → loss bounces or explodes; it never settles

a good default → Adam with learning rate 0.001, then let it shrink during training

Overfitting — memorizing is not understanding

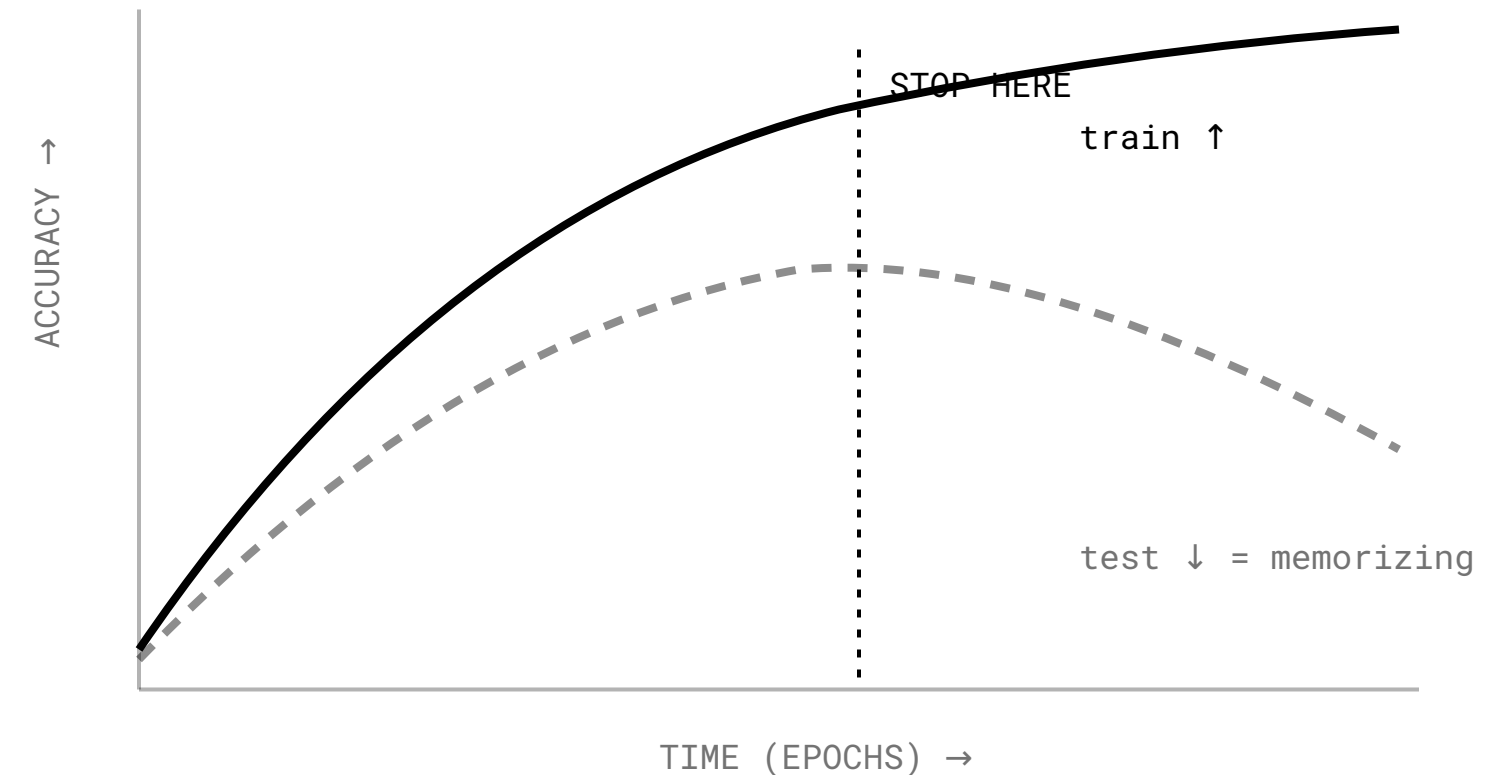
- A student who memorizes past papers aces every practice test — and fails the real exam. Networks do exactly this.
- So we **split the dataset**: TRAIN photos (study material) and TEST photos (the real exam — the network NEVER studies these).
- Watch both scores. Train accuracy climbs forever. Test accuracy rises... then **falls**. The fall is the moment memorizing started.
- Fixes: stop at the peak, get more varied data, or use the “forget a little on purpose” brakes below.

THE PROPER NAME

OVERFITTING — scoring high on study material, low on the real exam

TRAIN / TEST SPLIT — study material vs the never-seen exam

EARLY STOPPING · DROPOUT · WEIGHT DECAY — the standard anti-memorizing brakes



TURN THE KNOB

more epochs → train accuracy up forever; test accuracy up, then down

more varied data → memorizing starts later or never

Augmentation — free extra photos

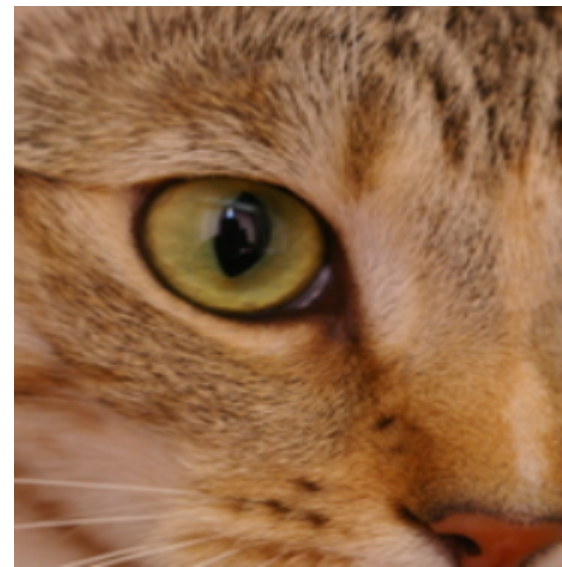
A flipped cat is still a cat. So every time a photo is used, change it a little — the network can never just memorize it.



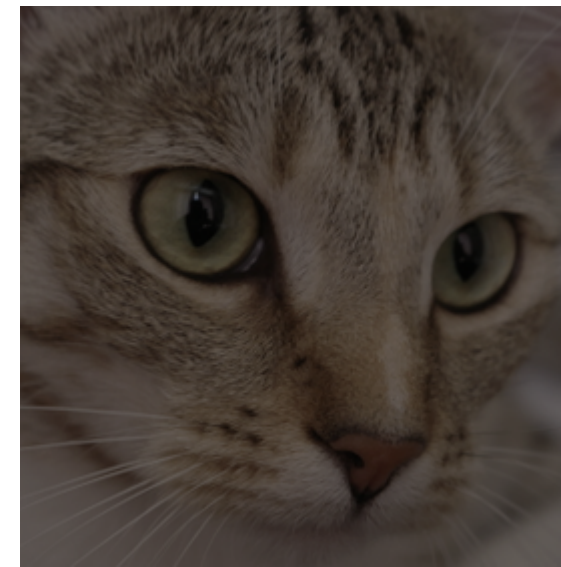
ORIGINAL



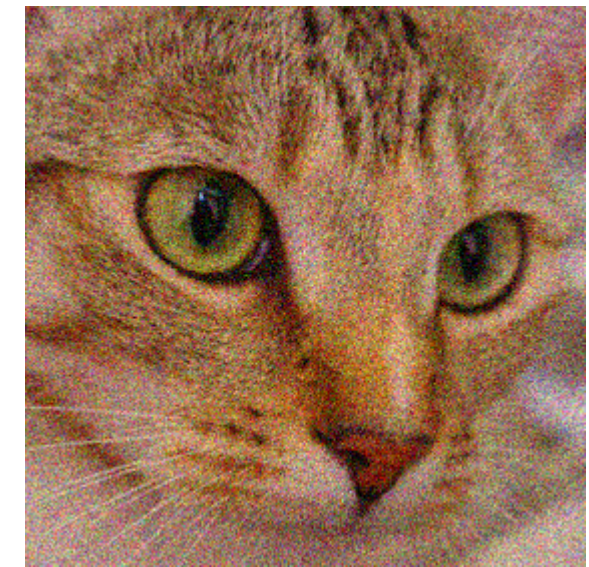
FLIPPED



CROPPED



RECOLORED



NOISY

```
train_tf = transforms.Compose([
    transforms.RandomCrop(32, padding=4),    # random framing
    transforms.RandomHorizontalFlip(),       # mirror half the time
    transforms.ToTensor(),
])
```

REAL LINES FROM THE TRAINING CODE THAT PRODUCED THE NUMBERS ON SLIDE 19

TURN THE KNOB

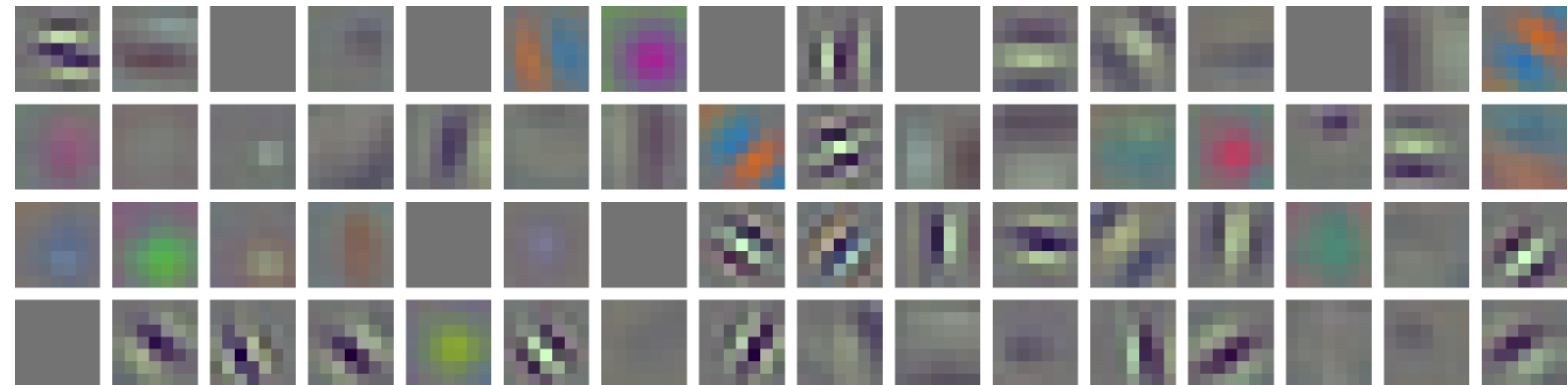
apply to TEST photos → never — the exam must stay fixed

too strong (crop 90% away) → the label itself is destroyed; accuracy drops

the classic trap → flipping digits turns 6 into 9 — every change must keep the label true

What the network actually taught itself

- These are the REAL first-layer stencils of ResNet18, a famous network trained on 1.28 million photos.
- **Nobody drew these.** They started as random noise.
- They became edge and color detectors — the network **discovered** that edges are the right place to start seeing.
- Layer 2 looks at edge-maps and finds textures: fur, stripes, mesh. Higher layers: ear, eye, wheel — then “cat”.



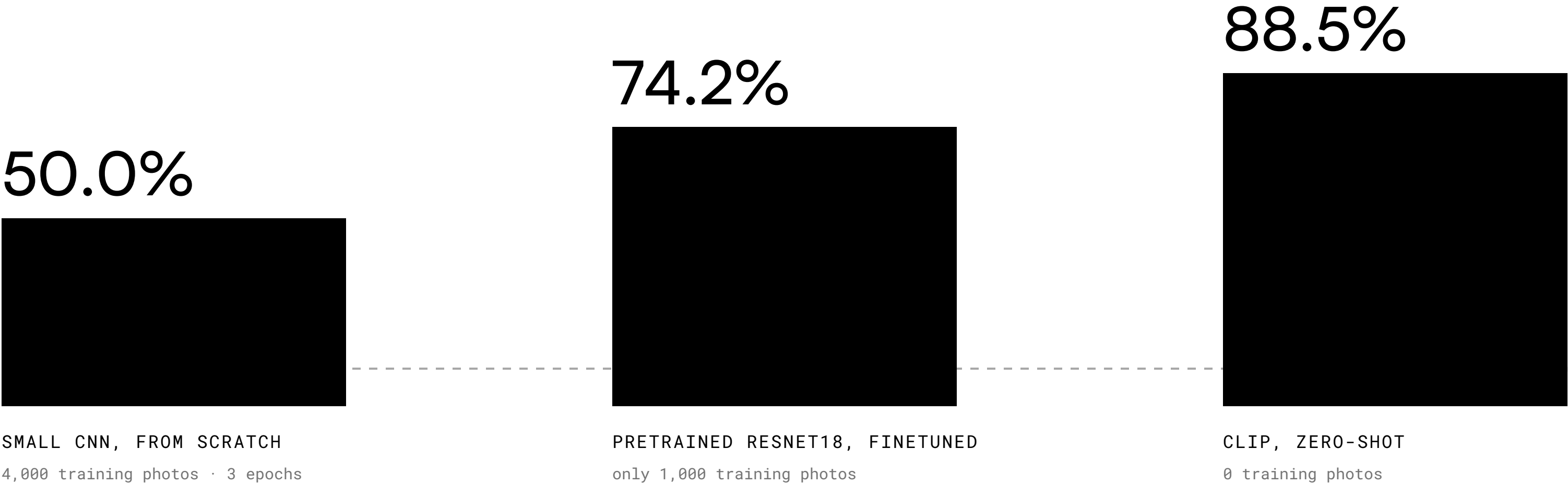
All 64 first-layer filters of ResNet18, enlarged. Compare with the stencil you computed by hand on slide 6.

EDGES → TEXTURES → PARTS → OBJECTS

the ladder every trained CNN climbs, layer by layer

Real numbers, run on an ordinary laptop CPU

Task: 10 kinds of small photos (CIFAR-10) — cat, ship, truck... Random guessing scores 10%. Every number below is reproducible from the notebooks in this repo.



The two bigger bars are Part 3 and Part 4 of this talk: borrowing trained eyes, and models that learned from the whole internet.

Never start from scratch.

Someone already spent weeks of computer time teaching a network to see, on 1.28 million photos. Those trained eyes are free to download.

Borrow trained eyes, swap the final decision

A chef who switches from Italian to Japanese food does not relearn chopping and seasoning — only the new recipes. Networks transfer the same way.

- Networks trained on ImageNet (1.28M photos, 1000 classes) already know edges, textures, fur, wheels — skills useful for ANY photo task.
- Chop off only the final decision layer, bolt on a fresh one for YOUR classes, train mostly that.
- That is how 1,000 photos beat 4,000 on slide 20: **74.2% vs 50.0%** — the eyes came pre-trained.
- Rule of thumb: with a pretrained start, 100—1,000 photos per class is often enough.

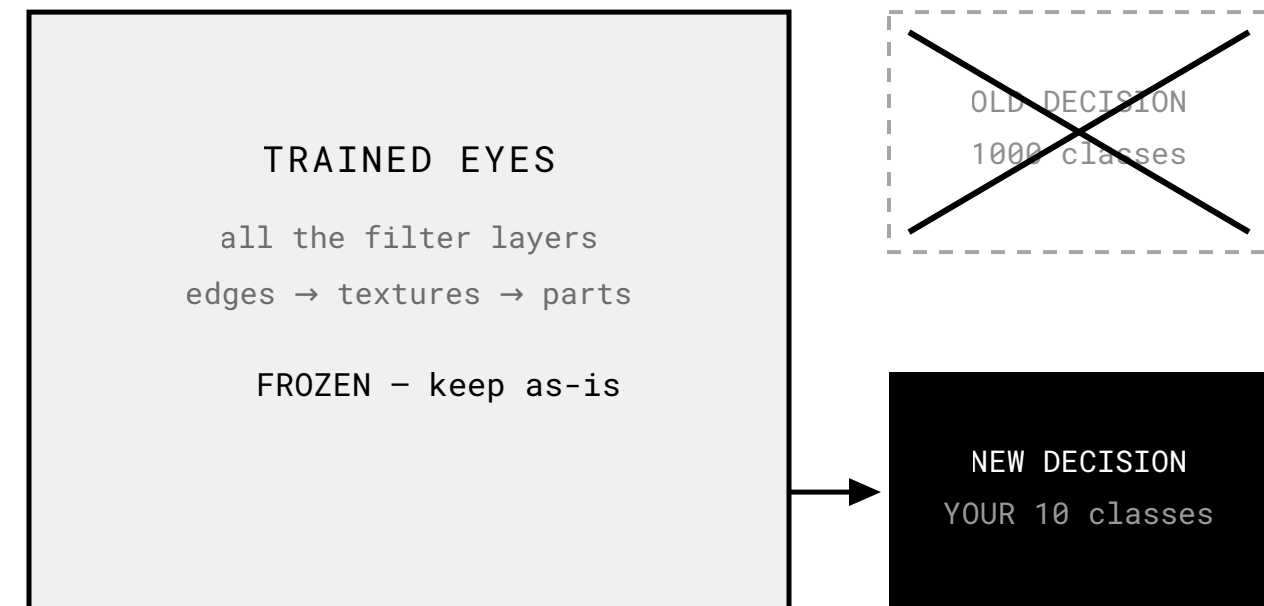
THE PROPER NAME

PRETRAINED MODEL / BACKBONE — a network someone already trained; the part you keep

TRANSFER LEARNING — reusing trained eyes on a new task

FINETUNING — the gentle retraining on your own photos

FREEZING — locking layers so training cannot change them

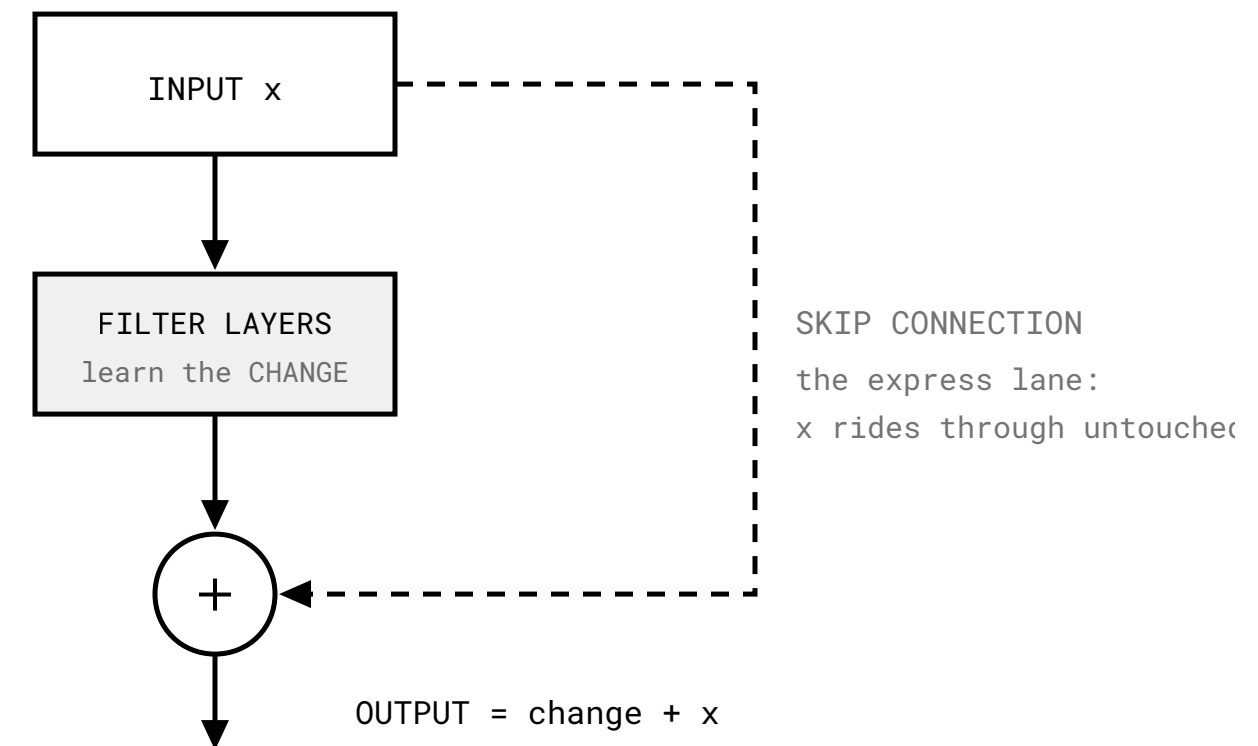


swap only the last piece, retrain the

ResNet — why “deeper” suddenly worked

Surprise from 2015: a 56-layer network scored WORSE than a 20-layer one. Deeper should never be worse — a deep net could just copy the shallow one. The fix was one humble arrow.

- In a deep stack, the learning signal must travel back through every layer — like a message whispered down a 50-person line, it **fades to nothing**.
- ResNet adds an express lane around every block: the input **skips ahead and is added back** at the end.
- Each block now only learns the small CHANGE it wants to make. Doing nothing is easy: $\text{change} = 0$.
- The learning signal rides the express lanes backwards too — 50, 100, 152 layers suddenly train fine.



THE PROPER NAME

SKIP / RESIDUAL CONNECTION — the express lane: $\text{output} = \text{input} + \text{learned change}$

RESNET-18 / -50 — the standard pretrained backbones; the number counts the layers

Transfer learning is four honest lines

This is the exact recipe behind the 74.2% bar — borrow, freeze, swap, train.

```
from torchvision import models

model = models.resnet18(weights="IMAGENET1K_V1") # 1. borrow trained eyes

for p in model.parameters():
    p.requires_grad = False                      # 2. freeze them

model.fc = nn.Linear(512, 10)                   # 3. swap the decision layer

# 4. train as usual -- only the new layer learns
```

RESULT ON OUR LAPTOP: 74.2% WITH 1,000 PHOTOS - VS 50.0% FROM SCRATCH WITH 4,000

TURN THE KNOB

unfreeze everything → more flexible, but with few photos it overfits — and a high learning rate DESTROYS the pretrained eyes

tiny dataset (<500 photos) → keep frozen — frozen usually WINS here

plenty of photos → unfreeze with a small learning rate (0.0001) for the backbone

weights=None → random start — you are back to the 50% bar

One label per photo was just the beginning.

Same trained eyes, different heads bolted on: find every object, cut them out pixel by pixel, talk to images in plain English, create images from nothing.

Object detection — WHERE, not just what



Real output of a detector (SSDLite) on our cat: **cat 97%**

- The model outputs a list: **box + label + confidence** for every object it can find — all in one pass over the photo.
- Messy detail: the raw model proposes MANY overlapping boxes per object. A cleanup step keeps each cluster's most confident box and deletes the rest.
- How do we grade a box? Overlap with the true box: **area of overlap ÷ area of union**. 1.0 = perfect, ≥ 0.5 traditionally counts as a hit.

THE PROPER NAME

BOUNDING BOX — the rectangle around an object

CONFIDENCE — how sure the model is about that box

IoU — INTERSECTION OVER UNION — the overlap score for grading boxes

NMS — NON-MAXIMUM SUPPRESSION — the duplicate-box cleanup step

TURN THE KNOB

confidence cutoff 0.5 → 0.8 → fewer boxes, fewer false alarms — but real objects get missed

NMS overlap limit down → stricter cleanup; two cats sitting together may merge into one box

Three detector families you should recognize

All reuse a pretrained backbone as their eyes — the difference is the head and the speed/accuracy trade.

FAMILY	THE IDEA, IN PLAIN WORDS	CHARACTER	PICK IT WHEN
YOLO “You Only Look Once”	cut the photo into a grid; every cell predicts boxes for objects centered in it — one single pass	the speed king — 150+ photos/second on a GPU	live video, robots, drones
SSD	same one-pass idea, but predicts from several map sizes at once — big maps catch small objects	fast and light — it produced the cat slide	phones and small devices
DETR	a transformer head asks ~100 learned questions: “is there an object like X anywhere?” — no grid, no NMS cleanup needed	elegant, accurate, slower to train	accuracy over speed, modern stacks

For the exam: know the names, the one-pass idea, and that YOLO = real-time. You do not need their internals.

Segmentation — a label for every pixel



PHOTO



MASK — EVERY PIXEL: CAT OR NOT



CUTOUT = PHOTO × MASK

- Boxes are rectangles; cats are not. Segmentation answers **per pixel**: which class does this pixel belong to?
- The classic shape is a letter **U**: shrink the photo to understand **WHAT** is there, then grow it back to full size to say **WHERE**, pixel-perfect.
- Shrinking loses fine detail — so express lanes copy crisp edges from the shrinking side straight to the growing side. Skip connections again!

THE PROPER NAME

SEGMENTATION MASK — the per-pixel answer sheet

SEMANTIC vs INSTANCE — “all cat pixels” vs “cat #1 vs cat #2”

U-NET / ENCODER-DECODER — the shrink-then-grow architecture with skip connections

WHERE YOU MET IT — background blur on video calls, medical scans, self-driving

Close the laptop lids — quick check

Answers out loud, no grades. If the room can answer these, we have earned the next part.

Why did 1,000 photos beat 4,000 photos in our experiment?

The 1,000-photo model started with pretrained eyes (transfer learning); the 4,000-photo model started from random noise.

What does a skip connection let a block do?

Learn only the small CHANGE to its input — and let the learning signal travel deep without fading.

Detection vs segmentation, one sentence each?

Detection: a box + label + confidence per object. Segmentation: a class for every single pixel.

What does NMS throw away?

Duplicate overlapping boxes around the same object — keeps the most confident one.

MYTH VS REALITY

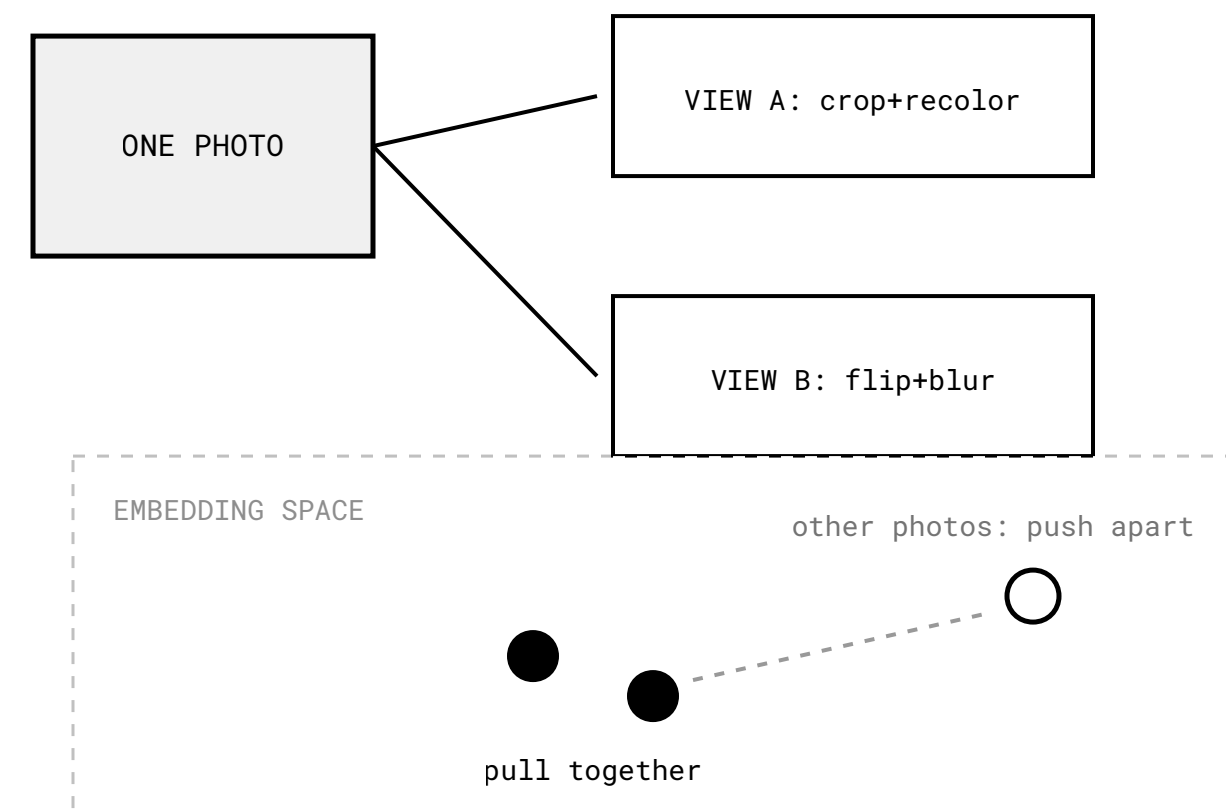
Myth: To use deep learning you need millions of photos and a giant computer.

Reality: With a pretrained backbone, 100—1,000 photos per class and a laptop is often enough. Our 74.2% ran on a CPU.

Self-supervised — learning without any labels

Labels are the expensive part: humans must tag every photo. The internet has billions of photos with no tags. Trick: make the photo its own label.

- Take a photo. Make **two different augmented views** of it (crop one, flip and blur the other) — slide 18's tricks, reused.
- Rule for the network: views of the **SAME** photo must land close together in embedding space; different photos far apart.
- To pull this off it must learn what **MATTERS** in an image — real seeing skills, no human labels anywhere.
- Then add a small decision layer with just a few labeled photos on top. Nearly free trained eyes.



THE PROPER NAME

SELF-SUPERVISED LEARNING — the data provides its own training signal
— no human labels

CONTRASTIVE LEARNING — the pull-together / push-apart recipe

CLIP — the model you can talk to

Don't teach it classes at all. Describe them in English and let the model pick the closest description.

- CLIP is TWO encoders: one turns photos into embeddings, one turns sentences into embeddings — in the SAME space.
- Trained on 400 million internet (photo, caption) pairs with the contrastive recipe from the last slide: photo and its caption pull together.
- To classify: embed the photo, embed each candidate sentence, pick the closest. **New classes = new sentences.** Zero retraining.
- That is the 88.5% bar from slide 20 — zero training photos.

THE PROPER NAME

CLIP — contrastive language—image pretraining — a shared space for photos and text

ZERO-SHOT — solving a task with no task-specific training examples

PROMPT — the sentence you classify with — wording matters

REAL CLIP OUTPUT ON OUR CAT PHOTO

a photo of a cat		98.4%
a photo of a tiger		1.6%
a photo of a dog		0.5%
a photo of a pizza		0.0%

TURN THE KNOB

rephrase the prompt → “a photo of a {{class}}” beats the bare word — a few % for free

add classes → just add sentences — no retraining

bigger CLIP (ViT-L/14) → ~63% → ~75% zero-shot on the 1000-class benchmark, ~3× slower

Zero-shot classification, complete and honest

```
import clip, torch

model, preprocess = clip.load("ViT-B/32")          # trained eyes + text eyes
labels = ["a photo of a cat", "a photo of a dog",
          "a photo of a tiger", "a photo of a pizza"]

image = preprocess(my_photo).unsqueeze(0)          # photo -> tensor
text = clip.tokenize(labels)                       # sentences -> tokens

img_emb = model.encode_image(image)                # both into the SAME space
txt_emb = model.encode_text(text)

sims = (img_emb @ txt_emb.T).softmax(dim=-1)        # closeness -> percentages
```

OUTPUT ON OUR CAT: CAT 97.9% · TIGER 1.6% · DOG 0.5% · PIZZA 0.0%

TURN THE KNOB

swap the labels list → an entirely new classifier, instantly

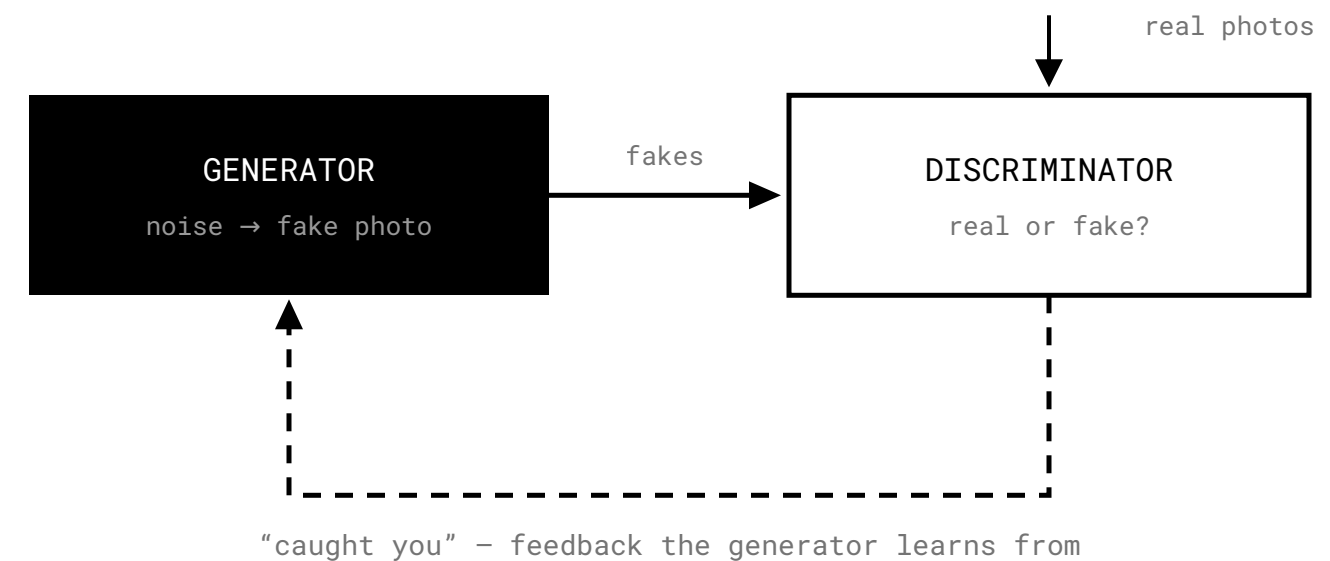
“cat” → “a photo of a cat” → matches how captions looked in training — accuracy up

ask something weird → “a photo of a grumpy cat” works too — it is just a sentence

GANs — a counterfeiter and a detective

Now the other direction: not understanding images — CREATING them.

- Two networks train against each other. The **counterfeiter** turns random noise into fake photos. The **detective** sees real photos and fakes, and calls real-or-fake.
- Every round the detective gets sharper, so the counterfeiter must get better — an arms race that ends in photorealistic fakes.
- Honest subtlety: the counterfeiter **NEVER** sees a real photo. It learns only from the detective's feedback.
- Famous, fast, but moody to train — sometimes the counterfeiter finds ONE face that always works and prints it forever.



both get better every round – until fakes pass inspection

THE PROPER NAME

GAN — GENERATIVE ADVERSARIAL NETWORK — the two-player forgery game

GENERATOR / DISCRIMINATOR — the counterfeiter / the detective

MODE COLLAPSE — the one-trick-forger failure mode

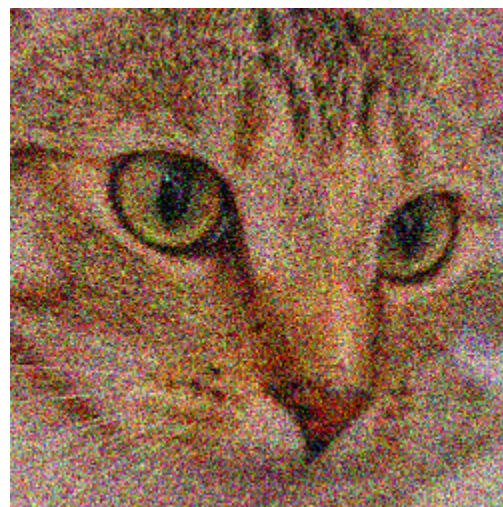
Diffusion — the idea behind modern image generators

DALL·E, Stable Diffusion, Midjourney — under the hood it is this. And you already know every ingredient.

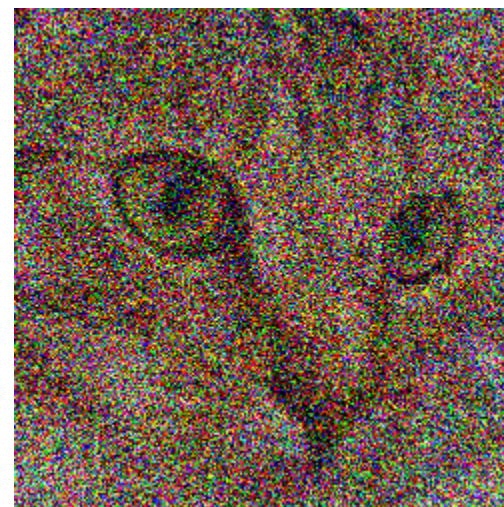
GENERATION RUNS RIGHT TO LEFT: START FROM PURE NOISE, REMOVE A LITTLE AT EACH STEP



CLEAN



STEP 250



STEP 500



STEP 750



PURE NOISE

- **Training:** take a photo, add a random amount of noise, ask a network to predict the noise. Easy, stable homework — no arms race.
- **Generating:** start from pure noise, remove a little ~50 times. A text prompt (via CLIP-style text understanding) steers every step.

THE PROPER NAME

DIFFUSION MODEL — learns to remove noise; generates by denoising step by step

GUIDANCE SCALE — how hard the prompt steers — ~7.5 is typical; too high looks burnt

STEPS — more = better and slower; 20—50 is common

Close the laptop lids — quick check

Answers out loud, no grades. If the room can answer these, we have earned the next part.

How does CLIP classify a photo of a thing it was never explicitly taught?

It embeds the photo and your candidate sentences into the same space and picks the closest sentence.

In a GAN, which network never sees a real photo?

The generator — it learns only from the discriminator's feedback.

What is a diffusion model's only skill?

Removing a little noise. Generation = applying that skill ~50 times starting from pure noise.

Self-supervised learning has no labels. What replaces them?

The photo itself: two augmented views must land close together in embedding space.

The IOAI syllabus map — where we covered what

Every Section 3 topic, the slide where it lives, and the one line to remember.

SYLLABUS TOPIC	SLIDES	ONE LINE TO REMEMBER
Images as tensors, pixels	05	a photo is 3 stacked grids of numbers 0—255
Convolution, filters, stride, padding	06–08	slide a stencil, multiply & add; stride = step size, padding = zero border
Pooling (max/avg/global)	09	keep each window’s winner — smaller map, shift-forgiving
Activations (ReLU)	10	zero the negatives — without the bend, depth is fake
CNNs & classification	11–12	stacked filters climb edges → textures → parts → objects
Loss, gradient descent, overfitting, augmentation	14–18	roll downhill on the wrongness landscape; never grade on the study set
Pretrained encoders, ResNet, transfer learning	21–23	borrow trained eyes, swap the decision layer; skips let depth train
Detection (YOLO/SSD/DETR), IoU, NMS	25–26	box + label + confidence in one pass; IoU grades, NMS dedupes
Segmentation, U-Net	27	a class per pixel; shrink to understand, grow back to localize
Self-supervised / contrastive	29	two views of one photo pull together — the data labels itself
CLIP & zero-shot	30–31	photos and sentences share one space; new classes = new sentences
Generative: GANs, diffusion	32–33	forgery arms race vs learn-to-denoise — diffusion won the quality war

Every word we earned today — all 30

pixel — one square of a photo, holding a number 0—255

tensor — a grid (or stack of grids) of numbers

model — a machine made of numbers: input → guess

label — the correct answer attached to an example

convolution — slide a stencil, multiply matching cells, add

filter / kernel — the stencil of numbers

feature map — the glow-map: bright = pattern found here

stride — how far the stencil steps each move

padding — zeros glued around the border

pooling — keep each window's max (or mean) — shrink the map

ReLU — negatives → 0; the bend that makes depth real

CNN — the full stack of filter layers

embedding — the short number-list summary of an input

softmax — scores → percentages that sum to 100

loss — the wrongness score

gradient descent — nudge every number downhill on the loss

learning rate — the step size of those nudges

epoch — one full pass through the dataset

overfitting — memorizing the study set, failing the exam

augmentation — flip/crop/recolor — free extra photos

pretrained / backbone — a network someone already trained

transfer learning — reuse trained eyes on a new task

finetuning — gentle retraining on your own data

skip connection — express lane: output = input + change

bounding box / IoU / NMS — detection's rectangle, grade, and dedupe

segmentation mask — a class for every pixel

self-supervised — the data provides its own training signal

CLIP / zero-shot — shared photo-text space; classify by sentence

GAN — counterfeiter vs detective

diffusion — generate by removing noise, step by step

The knob table — change this, get that

The practical round usually asks exactly this: predict what a change does before you run it.

TURN THIS KNOB	WHAT HAPPENS
more filters per layer (32 → 128)	richer patterns spotted; ~4× the numbers to learn and compute
bigger kernel (3×3 → 7×7)	sees wider patterns; 9 → 49 multiplies per stop
stride 1 → 2	output map halves; faster, blurrier view
remove ReLU	depth collapses to one layer; accuracy tanks
learning rate ×100	loss bounces or explodes — never settles
learning rate ÷100	crawls; may never arrive in time
train far past the test-accuracy peak	overfitting — train score up, exam score down
add flip/crop augmentation	typically +3—8% test accuracy on small datasets
pretrained start vs random start	our receipts: 74.2% with 1,000 photos vs 50.0% with 4,000
detector confidence cutoff up	fewer false alarms, more missed objects
better CLIP prompt wording	a few % accuracy for free — no retraining
diffusion guidance scale up (7.5 → 20)	obeys the prompt harder; colors go burnt, variety drops

It was never magic.
It was multiply, add,
and practice.

Everything today ran on an ordinary laptop, and every number on these slides is real. The notebooks to re-run them are in our repo.